

LANGGRAPH

LANGGRAPH

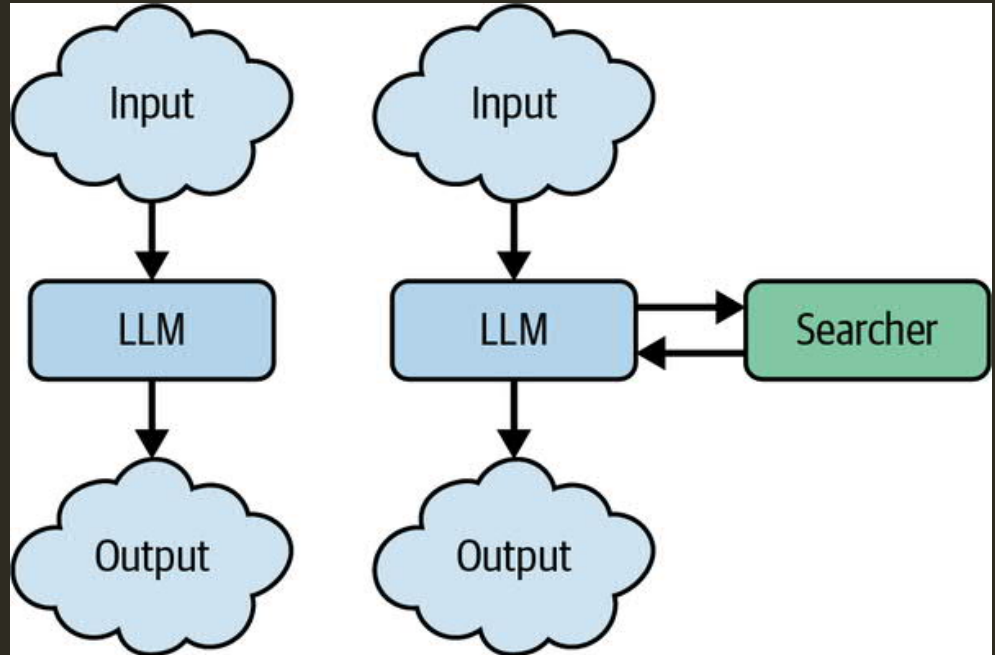
- Open source library authored by LangChain
- LangGraph was designed to enable developers to implement:
 - Multiactor
 - Multistep
 - Stateful
 - Cognitive architectures
- LangGraph models AI workflows as a graph instead of a chain.

LANGGRAPH (CONT...)

- LangGraph is a low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful agents.
- Very low-level, and focused entirely on agent orchestration!
- You don't need to use LangChain to use LangGraph
- Use LangChain's [agents](#) that provide prebuilt architectures for common LLM and tool-calling loops.
- LangGraph is inspired by [Pregel](#) and [Apache Beam](#).
- The public interface draws inspiration from [NetworkX](#).
- LangGraph is built by LangChain Inc, the creators of LangChain, but can be used without LangChain.

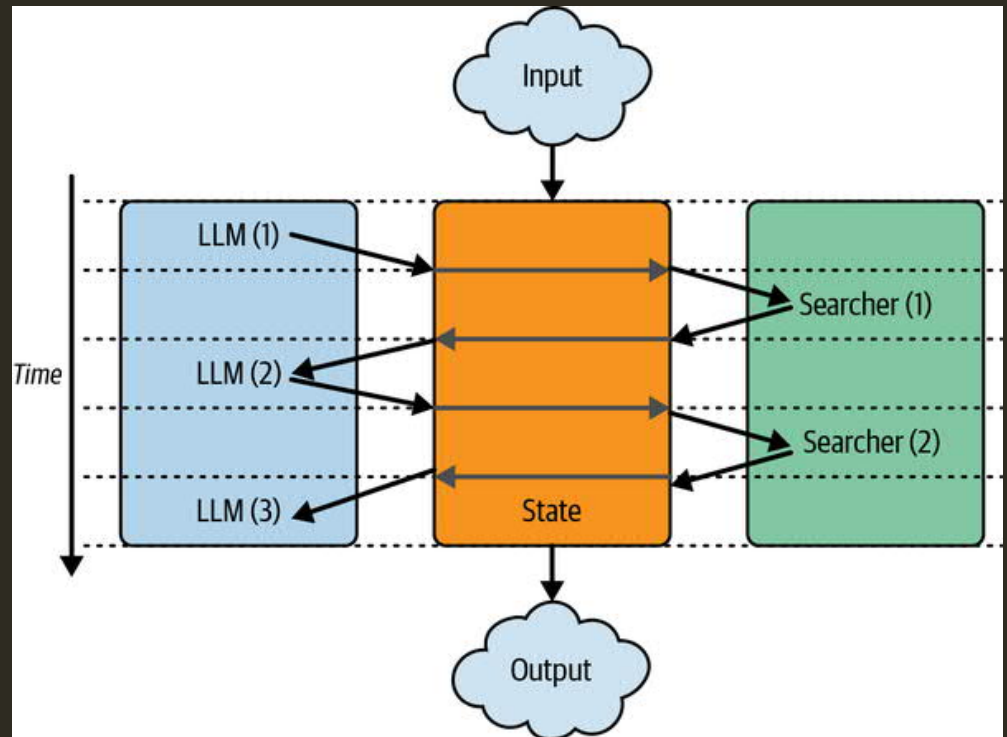
LANGGRAPH - MULTIACTOR

- A team of specialists can build something together that none of them could build alone.
- Similarly, an LLM prompt is much more powerful when paired up with related artifacts:
 - Prompts
 - Tools
 - Agents
 - MCP Servers, etc.



ACTOR COORDINATION

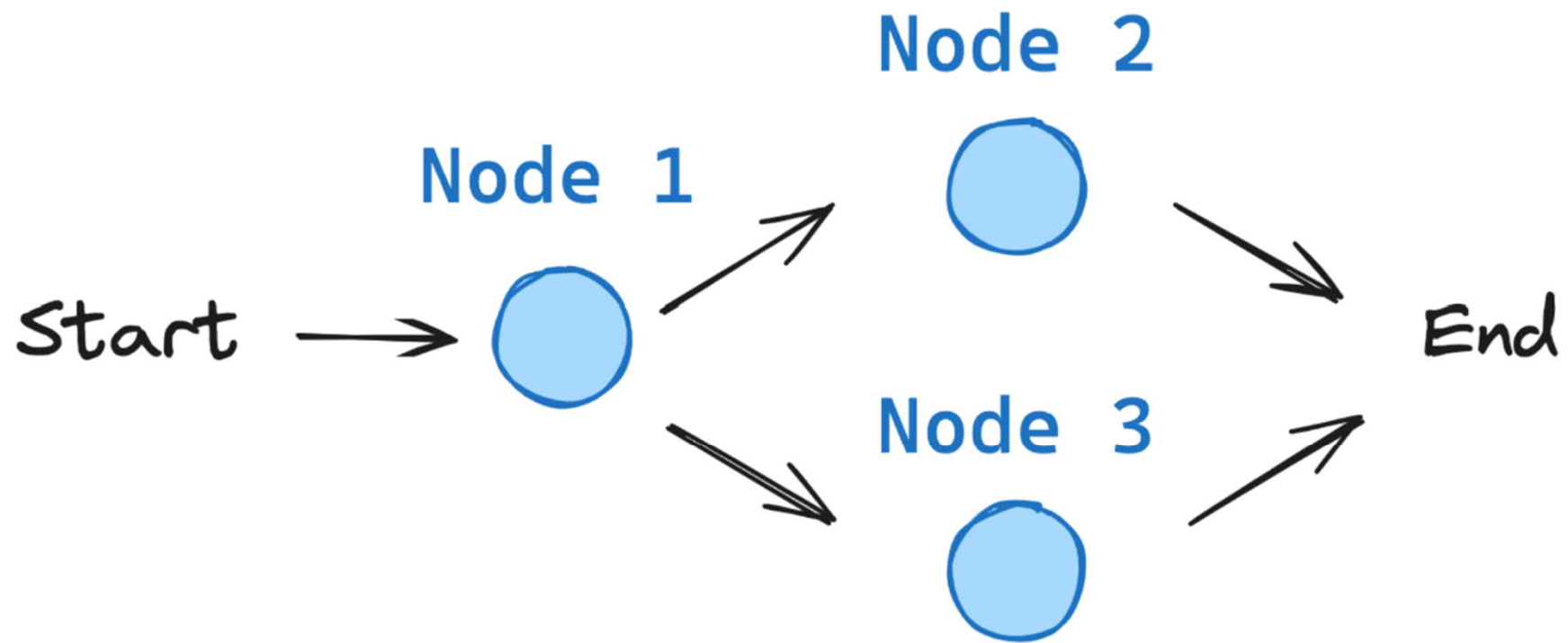
- An application with multiple actors needs a coordination layer to manage the following:
 - Define the actors involved (the nodes in a graph), and
 - How they hand off work to each other (the edges in that graph)
 - Schedule execution of each actor
- With a single central state, we can:
 - Snapshot and store the central state during or after each computation.
 - Pause and resume execution, which makes it easy to recover from errors.
 - Implement human-in-the-loop controls



GRAPH IN LANGGRAPH (DAG!)

- Each *graph* is then made up of the following:
 - *State*
 - The data received from outside the application, modified and produced by the application while it's running.
 - *Nodes*
 - Each step to be taken. Nodes are simply Python/JS functions (tools), which receive the current state as input and can return an update to that state (that is, they can add to it and modify or remove existing data).
 - *Edges*
 - The connections between nodes. Edges determine the path taken from the first node to the last, and they can be either:
 - fixed (that is, after Node B, always visit node D) or
 - conditional (evaluate a function to decide the next node to visit after node C).

IMPLEMENTING A BASIC GRAPH



GRAPH BUILDING BLOCKS

- Graph state (object to hold state of the workflow/graph)
- Nodes (python functions)
- Edges (node connection)
 - Holds logic to control the flow
 - Normal Edges: Always go from point A → point B
 - Conditional Edge: Logic (some python function) to decide which path to take!
 - Not really an edge!
 - START/END Nodes: Special nodes indicating start and end of the graph
 - Compile the graph (and optionally display to verify the structure is as intended!)

CREATING OUR FIRST GRAPH

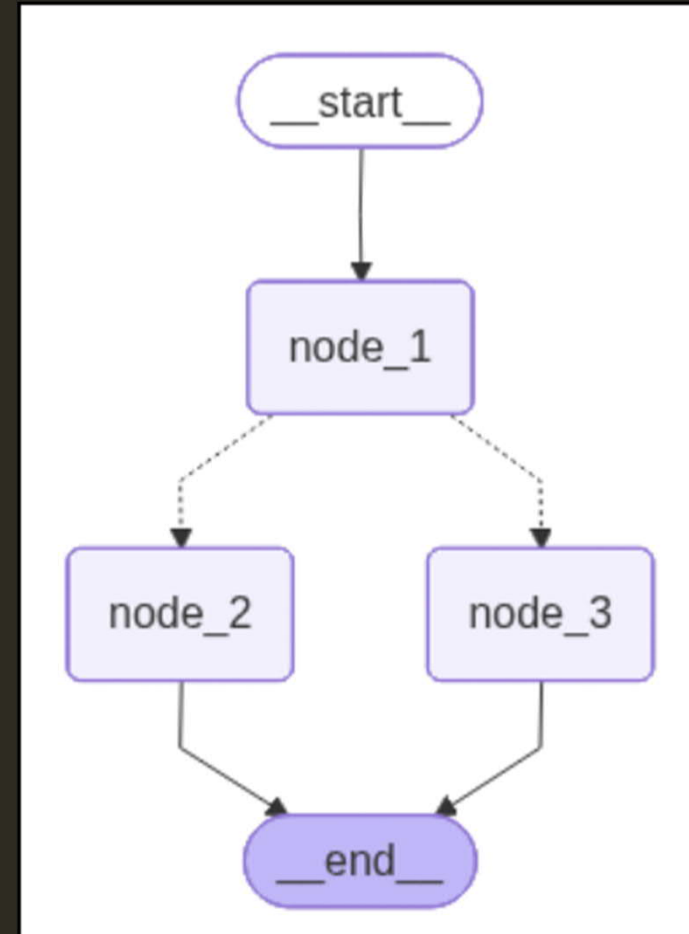
Build graph

```
builder = StateGraph(State)
builder.add_node("node_1", node_1)
builder.add_node("node_2", node_2)
builder.add_node("node_3", node_3)
```

Logic

```
builder.add_edge(START, "node_1")
builder.add_conditional_edges("node_1", decide_mood)
builder.add_edge("node_2", END)
builder.add_edge("node_3", END)
```

```
graph = builder.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```



CONDITIONAL EDGE

```
def decide_mood(state) -> Literal["node_2", "node_3"]:  
  
    # Often, we will use state to decide on the next node to visit  
    user_input = state['graph_state']  
  
    # Here, let's just do a 50 / 50 split between nodes 2, 3  
    if random.random() < 0.5:  
        # 50% of the time, we return Node 2  
        return "node_2"  
  
    # 50% of the time, we return Node 3  
    return "node_3"
```

DEMO (BASIC GRAPH)

Basic Graph

REDUCERS

- Reducers define *how simultaneous/competing updates are performed by multiple actors!*
- Imagine a graph with two branches running in parallel:
 - Branch A produces {"messages": [msgA]}
 - Branch B produces {"messages": [msgB]}
- Both update the same state key: messages, but there are multiple ambiguities:
 - Should the final state contain only msgA?
 - only msgB? both?
 - in what order?
- A reducer answers that question deterministically.
- Typically defined in the state schema, i.e., in the typed definition of your state
 - You are basically defining that when multiple updates happen, they should be done in a specific way!
- Reducers are crucial for parallelism

REDUCER PATTERNS

- Keep *all* values? → list append / set union
- Keep only one authoritative value? → last write wins
- Combine partial structured data? → dict merge
- Aggregate metrics? → sum/max/min
- Example: Message Accumulation
 - LangGraph lets you define the messages state key with a reducer that concatenates lists.
 - $\text{new_messages} = \text{old_messages} + \text{node1_messages} + \text{node2_messages}$
 - Done using the built-in `add_messages` reducer

DEMO (REDUCERS)

Basic Graph

MESSAGE REDUCER

```
from typing import Annotated
from langgraph.graph.message import add_messages

class MessagesState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
```

Type	Meaning
messages	The key name in the state dictionary
list[AnyMessage]	The type — a list of LangChain message objects (Human, AI, Tool, etc.)
add_messages	The reducer function — tells LangGraph how to update this key
Annotated[...]	Annotated is from Python's typing module. It lets you attach extra metadata to a type hint without changing the type itself. LangGraph reads that metadata at graph construction time and uses add_messages as the merge strategy whenever a node returns a new value for messages.

- *Without reducer:* messages becomes [new_message] — previous messages are lost.
- *With add_messages reducer:* new_message is appended to the existing list — conversation history is preserved!

REDUCER FOR MESSAGES: MESSAGESSTATE

- MessagesState is a pre-built conversation-oriented state schema meant to eliminate boilerplate when your graph state is basically “a growing list of messages.”
- Under the hood, message accumulation is typically done via the `add_messages` reducer, which “merges two lists of messages, updating existing messages by ID” and makes the state append-only by default.
- BUT, `add_messages` is NOT just append!
 - Besides merging two lists of messages, it also updates existing messages if a new message is provided by an existing ID!

```
msgs1 = [HumanMessage(content="Hello", id="1")]
msgs2 = [HumanMessage(content="Hello again", id="1")]
add_messages(msgs1, msgs2)
# [HumanMessage(content='Hello again', id='1')]
```


LANGGRAPH HELPERS

- In LangGraph, `tools_condition` is a prebuilt “routing” (conditional-edge) function used in tool-calling agent loops (ReAct-style). Its whole job is simple:
 - It looks at the last model (AI) message in your state.
 - If it contains `tool_calls`, route to the tools execution node;
 - Otherwise, end the graph.

```
tools_condition(state, messages_key="messages") →  
Literal["tools", "__end__"]
```

- Returns "tools" when the last message is an `AIMessage` and has one (or more) `tool_calls`
- Returns "__end__" when there are no tool calls, meaning the agent is ready to respond and the workflow should terminate

DEMO (ROUTER)

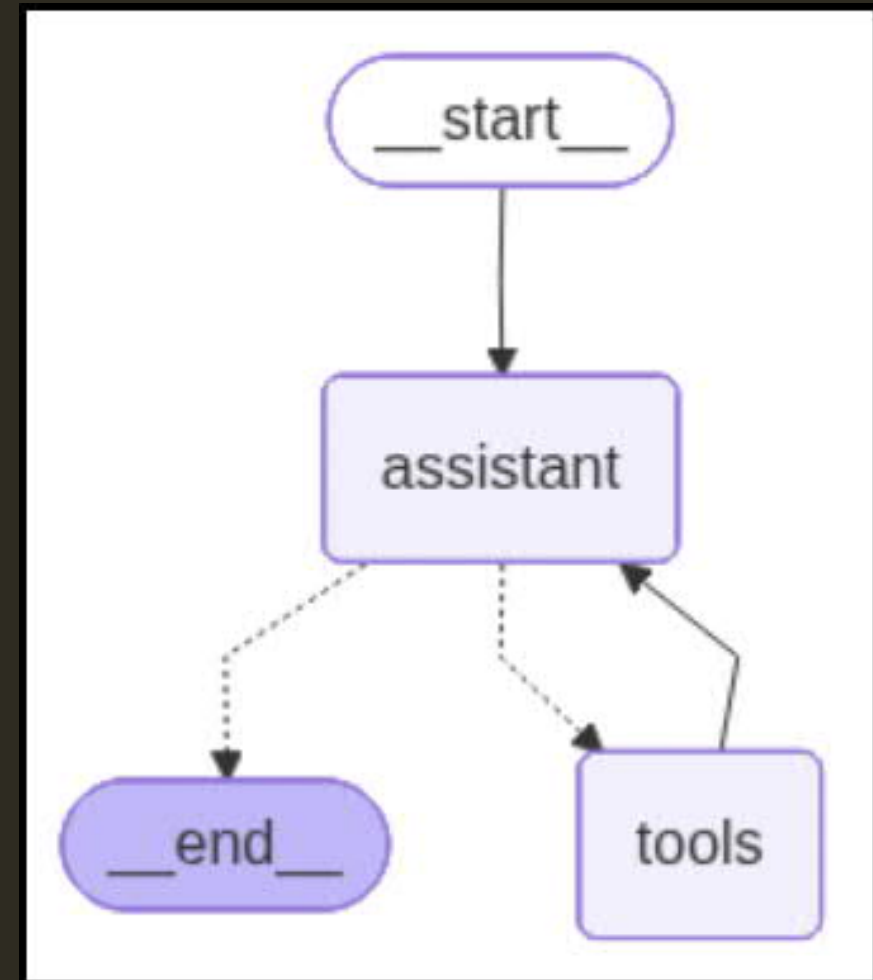
Router Implementation with Langgraph

AGENTS-LIKE STRUCTURE

LLM Tool Workflow in Langgraph

AGENTS IN LANGGRAPH

- Assistant Node contains our LLM with Tools
- Based on invocation, if there is a tool call, tool is invoked, and response is sent to conditional edge
- Condition contains tools_condition. If the response is tools, flow is sent to assistant node again.
- If not, graph ends!



DEMO (AGENTS)

Langgraph Agents

DEMO (AGENTS WITH MEMORY)

Agents with Memory

SUBGRAPHS

SUBGRAPHS

- A subgraph is simply another graph used as a node inside a larger LangGraph.
 - In LangGraph, nodes are just functions or components that take state, perform work, and return updated state.
- As nodes can be arbitrary functions (not just LLM calls), a compiled graph can also act as a node in another graph, which creates a hierarchical workflow.
- Distributing development: Allow your different teams to work on different parts of the graph independently

SUBGRAPH PATTERNS

Pattern	When to use	State schemas
<u>Call a subgraph inside a node</u>	Parent and subgraph have different state schemas (no shared keys), or you need to transform state between them	You write a wrapper function that maps parent state to subgraph input and subgraph output back to parent state
<u>Add a subgraph as a node</u>	Parent and subgraph share state keys—the subgraph reads from and writes to the same channels as the parent	You pass the compiled subgraph directly to <code>add_node</code>

INVOKING SUBGRAPH INSIDE A NODE

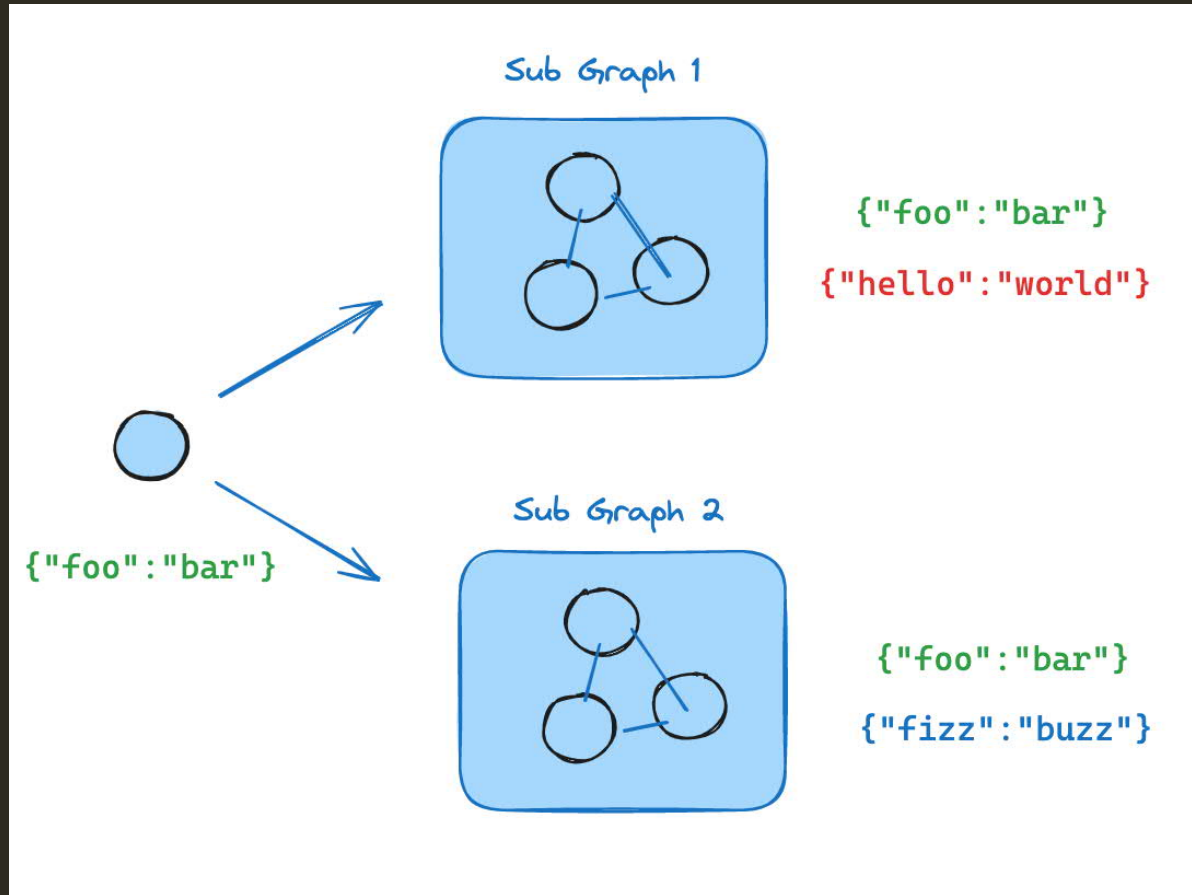
- When the parent graph and subgraph have different state schemas (no shared keys), invoke the subgraph inside a node function. This is common when you want to keep a private message history for each agent in a multi-agent system.
- The node function transforms the parent state to the subgraph state before invoking the subgraph, and transforms the results back to the parent state before returning.

```
class SubgraphState(TypedDict):  
    bar: str  
  
# Subgraph  
def subgraph_node_1(state: SubgraphState):  
    return {"bar": "hi! " + state["bar"]}
```

```
# Parent graph  
class State(TypedDict):  
    foo: str  
  
def call_subgraph(state: State):  
    # Transform the state to the subgraph state  
    subgraph_output = subgraph.invoke(  
        {"bar": state["foo"]})  
  
    # Transform response to the parent state  
    return {"foo": subgraph_output["bar"]}
```

ADD SUBGRAPH AS NODE

- When the parent graph and subgraph share state keys, you can pass a compiled subgraph directly to `add_node`.
- The subgraph reads from and writes to the parent's state channels automatically. For example, in multi-agent systems, the agents often communicate over a shared messages key.



SUBGRAPH AS NODE: EXAMPLE

```
sub_builder = StateGraph(State)

def node1(state):
    ...
def node2(state):
    ...

sub_builder.add_node("node1", node1)
sub_builder.add_node("node2", node2)
sub_builder.add_edge(START, "node1")
sub_builder.add_edge("node1", "node2")

subgraph = sub_builder.compile()
```

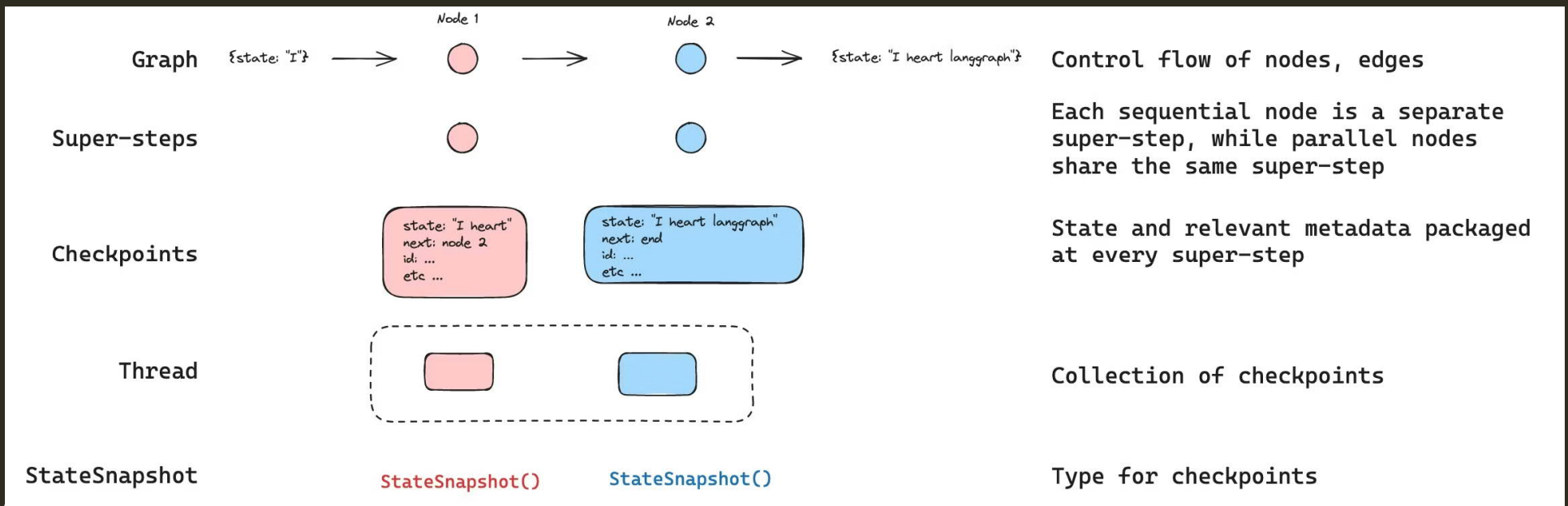
```
main_builder = StateGraph(State)

main_builder.add_node("analysis", subgraph)
main_builder.add_edge(START, "analysis")

main_graph = main_builder.compile()
```

LANGRAPH - PERSISTENCE

- LangGraph has a built-in persistence layer that saves graph state as checkpoints.
- When you compile a graph with a checkpointer, a snapshot of the graph state is saved at every step of execution, organized into threads.



LANGGRAPH - DURABILITY

- Durable execution is a technique in which a process or workflow saves its progress at key points, allowing it to pause and later resume exactly where it left off.
- Enable persistence in your workflow by specifying a checkpointer that will save workflow progress.
- Specify a thread identifier when executing a workflow. This will track the execution history for a particular instance of the workflow.
- Wrap any non-deterministic operations (e.g., random number generation) or operations with side effects (e.g., file writes, API calls) inside tasks to ensure that when a workflow is resumed, these operations are not repeated

TASKS

```
def call_api(state: State):  
  
    """Example node that makes an API request."""  
    result = requests.get(state['url']).text[:100]  
  
    return { "result": result }
```

```
@task def _make_request(url: str):  
  
    """Make a request."""  
    return requests.get(url).text[:100]
```

DEPLOYMENT

LANGSMITH DEPLOYMENT

- LangSmith Deployment is a workflow orchestration runtime purpose-built for agent workloads.
- It take your agent code (for example a LangGraph graph) and expose it as a production-ready API service with built-in state management, scaling, monitoring, and persistence.
- LangSmith Deployment is **framework-agnostic**: you can deploy agents built with LangGraph or other frameworks.
- LangSmith's deployment layer provides:
 - Durable execution of agent workloads with persistent state & conversation memory
 - API endpoints for interacting with agents
 - Monitoring and debugging tools
 - Deployment infrastructure (cloud, hybrid, or self-hosted)

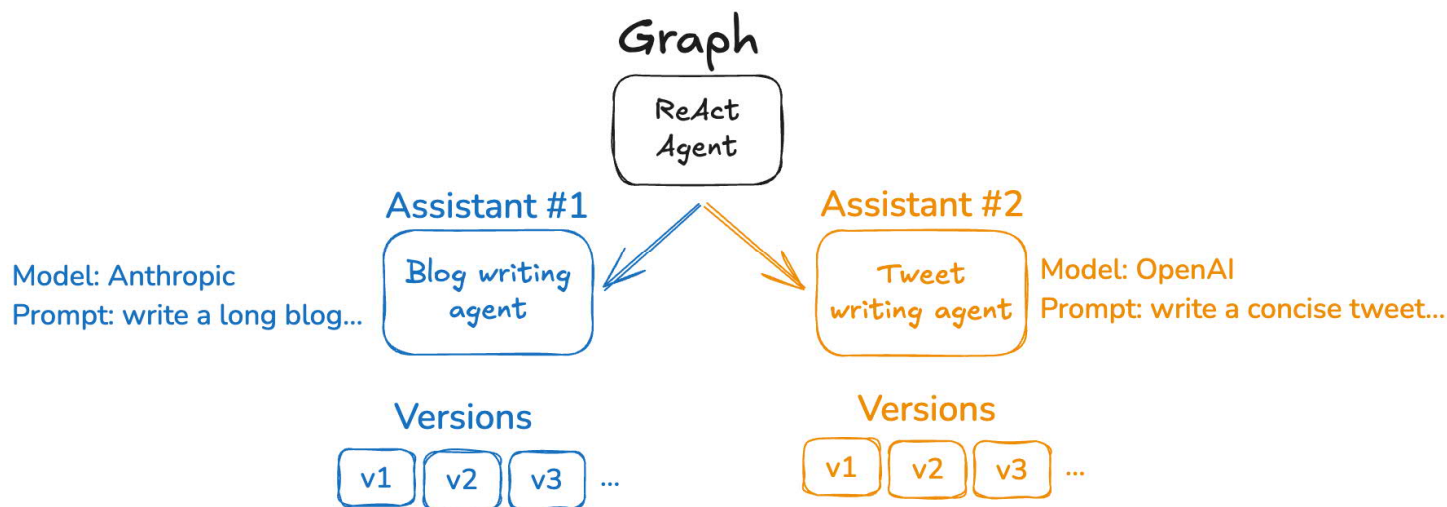
AGENT SERVER

- When you deploy to LangSmith, you are basically deploying to an Agent Server — a backend service designed to execute agent graphs.
- Agent server exposes a structured API for managing agents and their executions.
- Agent Server API is built around four main concepts:

Entity	What it represents
Assistant	a configured agent
Thread	an execution context with shared state
Run	one execution of the agent
Store	persistent key-value memory

AGENT SERVER: ASSISTANT

- *Assistants* are an Agent Server concept that allow you to manage configurations (e.g., prompts, LLM selection, tools) separately from your graph's core logic.
- This enables you to create multiple, specialized versions of the same graph architecture with different behavior at runtime.



ASSISTANT

- An assistant is just an *instance* of a graph with a specific configuration.
- Multiple assistants can reference the same graph but can contain different configurations (e.g. prompts, models, tools).

```
assistant = await client.assistants.create(  
    "writing_agent",  
    context={"model_name": "openai"},  
    name="Blog Writer"  
)  
  
result = await client.runs.wait(  
    thread["thread_id"],  
    assistant["assistant_id"],  
    input={  
        "messages": [ ... ]  
    },  
)
```

LANGGRAPH STORE

- Store is the long-term memory layer: a persistent key-{value | document} store that survives across threads and conversations, unlike thread state/checkpoints which are scoped to runs or threads.
- Agent Server stores long-term memory in PostgreSQL by default and can use a custom store implementation instead.
- When you deploy on Agent Server, the server injects the configured store into graph execution. Completely decoupled from the code.
- **Do not configure the store in your graph code** for Agent Server, because the server manages it for runtime operations.

LANGGRAPH STORE

```
# Writing to store
namespace = ["users", "user123"]
key = "profile"

# Put / upsert a document
await client.store.put_item(
    namespace,
    key,
    {
        "name": "Ahmed",
        "preferences": {
            "tone": "concise",
            "language": "en"
        }
    }
)
```

```
# Read it back

item = await client.store.get_item(namespace, key)
print("GET:", item)

# Search under a namespace prefix
results = await client.store.search_items(
    ["users"],
    filter={"preferences.language": "en"},
    query="concise tone"
)

print("SEARCH:", results)

# List namespaces
namespaces = await client.store.list_namespaces()

print("NAMESPACES:", namespaces)
```

LANGGRAPH STORE

- Long-term memory is stored as JSON documents under a namespace + key model, and search can work within namespace prefixes with filters and semantic ranking.
- A very good pattern is to include tenant ID in the namespace, so memories don't leak across users.
 - ("tenant_a", "customers", "c42")
- This gives you:
 - data isolation
 - cleaner search boundaries
 - easier multi-tenant enforcement.

LANGGRAPH.JSON

- langgraph.json is the configuration file that defines how a LangGraph application should be built and run by the LangGraph CLI / Agent Server. In practice, it tells the runtime:
 - which graphs exist
 - how to load them
 - what dependencies to install
 - how to configure the server and runtime
- At runtime, the LangGraph CLI starts an Agent Server that runs graphs. This server “exposes API endpoints for runs, threads, assistants, etc.” and requires a configuration file (langgraph.json) describing graphs, dependencies, and environment variables.

LANGGRAPH.JSON

```
{  
  "dependencies": ["./", "langchain_openai"],  
  "graphs": {  
    "research": "./my_agent/research.py:graph",  
    "sql": "./my_agent/sql.py:graph",  
    "writer": "./my_agent/writer.py:graph",  
    "supervisor": "./my_agent/supervisor.py:graph"  
  },  
  "env": "./.env"
```

DEPENDENCIES

- dependencies tells the LangGraph CLI / Agent Server what packages or local code it needs to install so your graph can import and run successfully
- When you run commands like `langgraph dev`, `langgraph build`, or `langgraph up`, the CLI reads `langgraph.json` and uses dependencies to prepare the runtime environment for your graph server.
- Basically, it determines what Python packages and local project code should be installed inside the server environment before your graph is loaded.

DEPENDENCIES

- A single period "."
This tells the CLI to look for and install the current local Python package/project.
- A local directory path such as "./" or "./my_package"
This points to the directory that contains your pyproject.toml, setup.py, or requirements.txt. The docs explicitly say you should point to the directory, not the file itself.
- A Python package name such as "langchain_openai"
This installs a package from the Python environment/package index so your graph can import it.

LANGGRAPH CLI

Command	What it does
<u>langgraph dev</u>	Starts a lightweight local dev server, ideal for rapid testing.
<u>langgraph build</u>	Builds a Docker image of your LangGraph API server for deployment.
<u>langgraph deploy</u>	Builds and deploys a LangGraph image directly to LangSmith Deployments in a single step.
<u>langgraph dockerfile</u>	Emits a Dockerfile derived from your config for custom builds.
<u>langgraph up</u>	Starts the LangGraph API server locally in Docker. Requires Docker running; LangSmith API key for local dev; license for production.

LANGSMITH LOCAL DEVELOPMENT SERVER

```
c:\>langgraph dev
```

```
INFO:langgraph_api.cli:
```

Welcome to

LANGGRAPH

- 🚀 API: <http://127.0.0.1:2024>
- 🎨 Studio UI: <https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024>
- 📖 API Docs: <http://127.0.0.1:2024/docs>

This in-memory server is designed for development and testing.

For production use, please use LangSmith Deployment.